

Runnables in LangChain

1. Simple & Straightforward Notes

◆ Background

- Last video: Chains in LangChain → Sequential, Parallel, Conditional.
- Today: **Runnables** → foundation behind Chains.
- Without Runnables → you can't fully understand Chains.

◆ Why Runnables were needed?

- 2022: ChatGPT + OpenAI API released → boom in LLM-based apps.
- Problem:
 - Different companies (OpenAI, Anthropic, Google, etc.) had different APIs.
 - Developers struggled switching APIs.
- **LangChain's first solution:** Created components (wrappers) for LLM APIs → standard access with minimal code changes.

◆ Beyond LLMs: Building full apps needs more components

Example: PDF Q&A app needs →

1. Document Loader
2. Text Splitter
3. Embeddings generator
4. Vector Store + Retriever
5. Output Parser
6. LLM

LangChain created helper **components** for each step.

◆ Chains (before Runnables)

- Observed patterns:
 - **Prompt** → **LLM** → **Response** (common in every app).
 - Introduced **Chains** = connecting 2+ components into pipelines.
- Examples:
 - LLMChain (Prompt + LLM).
 - RetrieverQChain (Retriever + LLM).
 - Sequential Chains, SQL Chains, API Chains, Math Chains, etc.

✓ Advantage: Easy reuse of common workflows.

✗ Problem: Too many chains → heavy codebase, steep learning curve.

◆ Core Problem

- Components weren't standardized:
 - LLM → `.predict()`
 - PromptTemplate → `.format()`
 - Retriever → `.get_relevant_documents()`
 - Parser → `.parse()`
- Interfaces different → forced to write **custom functions (chains)** again and again.

◆ Solution → Runnables

- **Definition:** *A standard unit of work in LangChain.*
 - **Characteristics:**
 1. Takes **input** → returns **output**.
 2. All Runnables follow a **common interface** (`.invoke`, `.batch`, `.stream`).
 3. Can be **connected together** → output of one becomes input to next.
 4. A workflow of multiple Runnables itself is also a Runnable (recursive).
-

◆ Runnable Analogy = LEGO Blocks

1. Each block has a purpose (like each Runnable does one task).
 2. All blocks follow same interface (studs & holes = .invoke).
 3. Blocks can be connected in any order.
 4. A built structure itself behaves like a new block (modular & reusable).
-

◆ Code Illustration (Simplified)

Custom Components Before Runnables:

```
python

class FakeLLM:
    def predict(self, prompt):
        return "Some random response"

class FakePromptTemplate:
    def format(self, variables):
        return f"Write a poem about {variables['topic']}"
```

Problem → Different methods (predict, format).

Standardized with Runnables:

```
from abc import ABC, abstractmethod

class Runnable(ABC):
    @abstractmethod
    def invoke(self, input):
        pass

class FakeLLM(Runnable):
    def invoke(self, prompt):
        return {"response": "Delhi is the capital of India"}

class FakePromptTemplate(Runnable):
    def invoke(self, variables):
        return f"Write a poem about {variables['topic']}"
```

✅ Now both use .invoke → easy chaining.

Runnable Connector:

```
class RunnableConnector(Runnable):
    def __init__(self, runnables):
        self.runnables = runnables

    def invoke(self, input_data):
        for runnable in self.runnables:
            input_data = runnable.invoke(input_data)
        return input_data
```

Use Case:

```
prompt = FakePromptTemplate()
llm = FakeLLM()
chain = RunnableConnector([prompt, llm])

print(chain.invoke({"topic": "AI"}))
```

◆ Complex Example → Chain of Chains

- Chain 1: Generate a joke.
- Chain 2: Explain the joke.
- Final Chain: Combine both.

Shows recursive power → *Chains of Runnables*.

◆ Final Insights

- Runnables fixed the problem of **non-standard components**.
 - Made LangChain flexible, modular, and developer-friendly.
 - Future workflows = just connecting standardized blocks.
-

Perfect Rahul 👍

This video (**Video 9 – LangChain Runnables Part 2 | CampusX**) is all about **Runnable Primitives** and the new **LangChain Expression Language (LCEL)**.

I'll make your notes in **two parts**:

1. **Simple & Clear Notes (for revision)**
 2. **Creative & Engaging Notes (for deeper understanding)**
-

Runnables (LangChain Primitives + LCEL)

1. Recap (from Part 1)

- LangChain gave developers **components** (PromptTemplate, LLMs, Retrievers, Parsers, etc.).
 - Problem → each component had **different methods** (.format(), .predict(), .parse() ...).
 - Solution → Standardize with **Runnables** → all follow .invoke().
 - Benefit → Any component can connect with any other → flexible workflows.
-

2. Types of Runnables

1. Task-Specific Runnables

- Core LangChain components turned into Runnables.
- Examples: ChatOpenAI, PromptTemplate, StringOutputParser, Retriever.
- Each has a **specific purpose** (LLM = talk to models, Parser = clean output).

2. Runnable Primitives

- Special Runnables that **connect task-specific Runnables together**.
 - Help structure complex workflows (sequential, parallel, conditional).
 - Examples:
 - RunnableSequence
 - RunnableParallel
 - RunnablePassthrough
 - RunnableLambda
 - RunnableBranch
-

3. Runnable Primitives (Detailed)

◆ 3.1 RunnableSequence

- Connects **multiple Runnables sequentially**.

- Output of one = input of next.
- No limit on number of steps.

Code Example (Joke Generator + Explanation):

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StringOutputParser
from langchain.schema.runnable import RunnableSequence

prompt1 = PromptTemplate.from_template("Write a joke about {topic}")
prompt2 = PromptTemplate.from_template("Explain the following joke:\n{text}")

model = ChatOpenAI()
parser = StringOutputParser()

chain = RunnableSequence([prompt1, model, parser, prompt2, model, parser])
result = chain.invoke({"topic": "AI"})
print(result)
```

✓ Output → Explanation of the generated joke.

◆ 3.2 RunnableParallel

- Runs **multiple Runnables simultaneously**.
- Each receives the **same input**.
- Returns a dictionary of outputs.

Code Example (Tweet + LinkedIn Post):

```
from langchain.schema import RunnableParallel

parallel_chain = RunnableParallel({
    "tweet": RunnableSequence([prompt1, model, parser]),
    "linkedin": RunnableSequence([prompt2, model, parser])
})

result = parallel_chain.invoke({"topic": "AI"})
print(result["tweet"])
print(result["linkedin"])
```

✓ Output:

- "tweet" → AI-related tweet.
- "linkedin" → AI-related LinkedIn post.

◆ 3.3 RunnablePassthrough

- Special primitive → returns **input as output** (no processing).
- Use case → when you want to keep original data **alongside transformed data**.

Example: Joke + Explanation

- Normally → only explanation shows.
- With Passthrough → we also keep original joke.

```
from langchain.schema import RunnablePassthrough

parallel_chain = RunnableParallel({
    "joke": RunnablePassthrough(),
    "explanation": RunnableSequence([prompt2, model, parser])
})
```

✓ Output: Dictionary with both "joke" and "explanation".

◆ 3.4 RunnableLambda

- Converts a **Python function** into a Runnable.
- Useful for **custom logic** inside workflows (preprocessing, analysis).

Example (Word Counter):

```
from langchain.schema import RunnableLambda

def word_count(text: str) -> int:
    return len(text.split())

word_counter = RunnableLambda(word_count)
print(word_counter.invoke("AI makes life easier"))
```

✓ Output → 4 (number of words).

👉 Combine with LLM:

- Generate joke.
 - Run Passthrough → keep joke.
 - Run Lambda → count words in joke.
 - Return both joke & word count.
-

◆ 3.5 RunnableBranch

- Used for **conditional logic** (if/else).
- Like LangChain's "if-else statement".

Example (Report Summarizer):

```
from langchain.schema import RunnableBranch

branch_chain = RunnableBranch(
    (lambda x: len(x.split()) > 500,
     RunnableSequence([prompt2, model, parser])),
    (lambda x: True, RunnablePassthrough()) # default
)
```

✓ If report > 500 words → Summarize.

✓ Else → Return report as-is.

4. LangChain Expression Language (LCEL)

- New **syntax sugar** for defining chains.
- Instead of:

RunnableSequence([R1, R2, R3])

- You can write:

R1 | R2 | R3

- ✓ Simpler, cleaner, declarative.
 - ✓ Currently for RunnableSequence.
 - ✓ Future → may support Parallel (R1 & R2) and Branch.
-

5. Key Takeaways

- **Task-Specific Runnables** → core components (Prompt, LLM, Parser).
- **Runnable Primitives** → connect them into workflows.
 - Sequence = sequential
 - Parallel = simultaneous
 - Passthrough = keep input
 - Lambda = custom function
 - Branch = if/else logic
- **LCEL** makes chain creation easier: R1 | R2 | R3.
- Runnables = **building blocks** → combine them into powerful pipelines.